

Migración de **datos.js** a base de datos **MariaDB**

Proyecto Distribuido – Gestión de recursos sanitarios

Base de datos: **Practica** • Servidores: REST (3000), RPC (3501), WebSocket (3502)

Informe técnico de implementación

21 de septiembre de 2026

1. Objetivo

Sustituir el archivo **datos.js** (arrays en memoria) por una base de datos real **MariaDB (Practica)** como fuente de datos de los tres servidores del proyecto: REST, RPC y WebSocket. El objetivo es que los datos sean **persistentes** y estén **realmente compartidos** entre los tres servicios, sin cambiar el comportamiento visible para los clientes (gestor y sanitario).

2. La idea central del cambio

Antes, los datos vivían en la RAM de Node y se leían de forma **síncrona** (`recursos.find(...)` devolvía el resultado en la misma línea). Con la base de datos, los datos viven en **otro proceso** (MariaDB, puerto 3306) y hay que consultarlos por red: todo pasa a ser **asíncrono**. Este es el 90% del trabajo de la migración: convertir código síncrono en asíncrono.

datos.js (antes)	bd.js (ahora)
Exporta los datos (arrays)	Exporta la conexión (pool)
<code>datos.recursos</code>	<code>bd.query("SELECT ...")</code>
Estado compartido = array en RAM	Estado compartido = la base de datos
Se pierde al reiniciar	Persistente

3. Archivos creados y modificados

bd.js — NUEVO (sustituye a datos.js)

Crea el *pool* de conexiones a la base de datos y lo exporta con promesas (para poder usar `await` y `.then()`).

```
var mysql = require('mysql2');
var pool = mysql.createPool({
  host: 'localhost',
  user: 'Mangel',          // usuario con privilegios
  password: 'MiguelA.128', // su contraseña
  database: 'Practica',    // base de datos
  port: 3306,
  waitForConnections: true,
  connectionLimit: 10
});
module.exports = pool.promise();
```

servidor_rest.js — reescrito

Todas las rutas pasan a `async`, con consultas SQL preparadas (los `?` evitan inyección SQL) y `try/catch` para responder 500 si la BD falla. Ejemplo del login:

```
app.post("/api/gestores/login", async function (req, res) {
  try {
    var [filas] = await bd.query(
      "SELECT * FROM gestores WHERE user = ? AND pswd = ?",
      [req.body.login, req.body.password]
    );
    if (filas.length > 0) res.status(200).json(filas[0].id);
    else res.status(403).json("Login incorrecto");
  } catch (err) { res.status(500).json("Error en la base de datos"); }
```

```
});
```

servidor_rpc.js — reescrito (CAMBIO CRÍTICO)

Una consulta a la BD no puede devolverse con return inmediato. Por eso **obtenerCategorias** y **obtenerModelos** pasaron de registerSync a registerAsync (ahora reciben callback). Es el error más fácil de cometer y no da error visible: simplemente devolvería undefined.

```
// ANTES (sincrono)                                // AHORA (asincrono con callback)
function obtenerCategorias() {                      function obtenerCategorias(callback) {
  return categorias;                                bd.query("SELECT * FROM categorias")
}                                                    .then(([filas]) => callback(filas))
app.registerSync(obtenerCategorias);                .catch(() => callback([]));
                                                    }
                                                    app.registerAsync(obtenerCategorias);
```

servidor_ws.js — adaptado

La única parte que leía datos (la rama de avisos de tipo 'reserva') ahora consulta la BD de forma asíncrona; todo el reparto de avisos (*fan-out*) se hace dentro del .then(). Las ramas de recurso (azul) y reseña (verde) no tocan datos, así que no se modificaron.

```
bd.query("SELECT sanitario FROM reservas WHERE recurso = ? AND fecha_fin IS NULL",
  [msg.idRecurso])
  .then(function(resultado) {
    var reservasActivas = resultado[0];
    // ... construir texto y enviar avisos a los sanitarios afectados ...
  });
```

4. Correcciones en los clientes

Con **datos.js** los IDs eran texto ("R001", "TER1"). En la base de datos son **números** (AUTO_INCREMENT). El value de un <select> siempre es texto ("3"), pero el ID de la BD es número (3), por lo que la comparación estricta === fallaba. Se corrigieron dos comparaciones cambiando === por == (comparación laxa):

- cliente_rpc/main.js (filtrado de modelos por categoría)
- cliente_rest/main.js (nombre del modelo en el aviso WebSocket)

5. Problema resuelto en MariaDB

Al arrancar, MariaDB **rechazaba** al usuario Mangel (Access denied ... using password: YES). El motivo: existía Mangel@'%' pero también un **usuario anónimo** '@'localhost' que, al conectar desde localhost, tiene prioridad (localhost es más específico que %) y hacía fallar la autenticación.

La solución fue crear explícitamente Mangel@'localhost' con su contraseña y privilegios sobre Practica:

```
CREATE USER IF NOT EXISTS 'Mangel'@'localhost' IDENTIFIED BY 'MiguelA.128';
GRANT ALL PRIVILEGES ON `Practica`.* TO 'Mangel'@'localhost';
FLUSH PRIVILEGES;
```

Importante: este usuario vive en tu MariaDB actual. Si reinstalas XAMPP o mueves el proyecto a otro ordenador, deberás volver a ejecutar ese SQL (como root, en phpMyAdmin).

6. Pruebas realizadas (todas correctas)

Servicio	Prueba	Resultado
REST	GET categorias / modelos / ubicaciones	Datos reales OK
REST	Login gestor correcto / incorrecto	id / HTTP 403
REST	GET recursos con filtro por categoria	Filtrado OK
REST	GET reservas y resenyas de un recurso	OK
REST	POST + DELETE de recurso	201 / 200
RPC	obtenerCategorias / obtenerModelos (ex-Sync)	OK
RPC	loginSanitario, obtenerRecursos, tiempoPendiente	OK
RPC	reservar -> iniciar -> finalizar -> cancelar	Ciclo OK
RPC	crearResenya	OK
BD	Recuento final tras pruebas	30 / 18 / 10, sin basura

7. Cómo arrancar el proyecto

Con XAMPP (MySQL) encendido:

```
cd /Users/miguelangel/Desktop/ProyectoTelemedicina/ProyectoDistribuido-1
node app/servidor.js
```

- Cliente gestor (REST): <http://localhost:3000/appCliente/>
- Cliente sanitario (RPC): <http://localhost:3000/appSanitario/>

Nota: el archivo datos.js se conserva como respaldo, pero ya no lo importa ningún servidor.